BẢN CHẤT HỆ THỐNG 8D COPILOT: LOGIC AI & CƠ SỞ DỮ LIỆU

> **Dành cho thành viên đội thi UIT_4Lovpe**
> *Tài liệu này được viết bằng văn nói đời thường của dân kỹ thuật, giải thích bản chất "tại sao lại làm như vậy", không dùng từ ngữ sáo rỗng để bạn đọc là hiểu ngay và tự tin thuyết trình trước Ban Giám Khảo (BGK).*

---

0. NÓI MỘT CÂU DUY NHẤT VỀ BẢN CHẤT DỰ ÁN

Nếu Ban Giám Khảo hỏi: "*Tóm lại hệ thống của các bạn là cái gì? Có phải chỉ là gắn con ChatGPT vào để viết báo cáo không?*"

👉 **Bạn trả lời ngay bằng văn nói như sau:**

> *"Dạ thưa BGK, 8D Copilot **không phải là con chatbot gõ chữ tự do**. Nó là một **Hệ Thống Trợ Lý & Trọng Tài Kỹ Thuật**:
>
> 1. Nó suy luận dựa trên **số đo dung sai vật lý** của máy móc chứ không nghe suy đoán cảm tính.
> 2. Nó tìm tiền lệ sự cố theo **mạng lưới thực thể (GraphAI)** chứ không tìm theo từ ngữ bề mặt.
> 3. Nó **tuyệt đối không bao giờ tự động bấm Approve** thay con người, vì trong ngành ô tô, chữ ký chịu trách nhiệm pháp lý bắt buộc phải là của Kỹ sư trưởng."*

---

1. BẢN CHẤT CƠ SỞ DỮ LIỆU (DATABASE) — TẠI SAO LẠI CÓ 16 BẢNG?

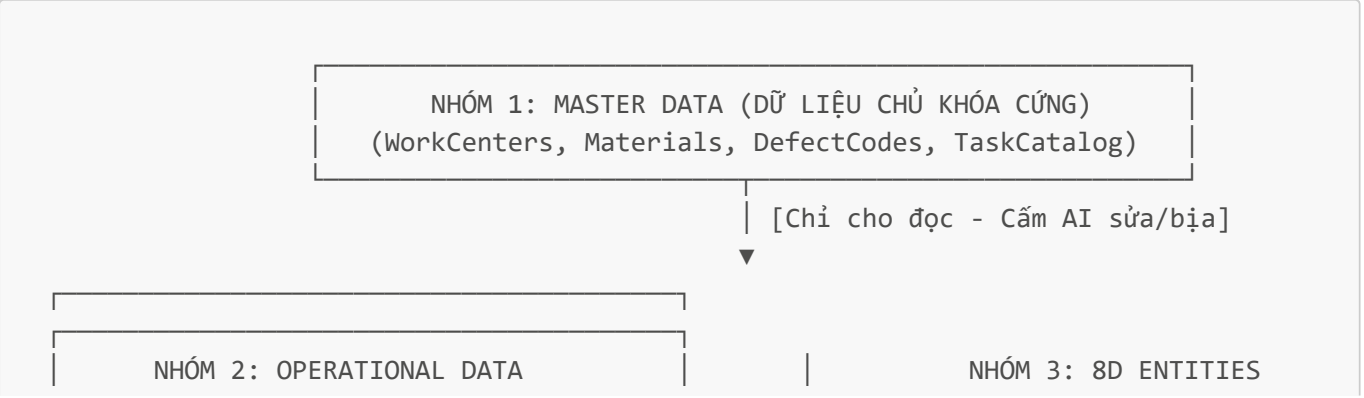
1.1. Nỗi đau ngoài xưởng máy

Khi một sự cố xảy ra ở xưởng (ví dụ phôi bị bavia hay xước), dữ liệu từ hệ thống SAP QM thường rất **"bẩn"**:

- Kỹ sư người Đức gõ số thập phân bằng dấu phẩy: `0,32 mm` (code bình thường parse là lỗi ngay).
- Mã vật tư bị thừa khoảng trắng: `" MAT-10247 "`.
- Mô tả lỗi viết tắt lung tung: "*Grat an Flanschkante*" (Bavia ở cạnh bích).

Nếu ném thẳng đống chữ bẩn này cho AI gõ tự do, nó sẽ bị **ảo giác (Hallucination)**: nó sẽ tự bịa ra một mã máy lạ hoắc hoặc tự chế ra một vật liệu không có trong nhà máy.

1.2. Cách giải quyết: Chia 16 bảng thành 3 nhóm rõ ràng



```
┌──────────────────────────────────────────┐
│                                          │
│        (DỮ LIỆU HIỆN TRƯỜNG)             │ JOIN │       (HỒ SƠ 8 BƯỚC XỬ LÝ)
│                                          │
│  • Notifications (Bản tin lỗi SAP)       ├──────►│  • Reports & Disciplines (D1 -> D8)    │
│  • Inspections (Số đo thước cặp, bar)    │       │  • IshikawaCauses (Xương cá 5M+E)      │
│  • Containments (Biên bản nhốt hàng)     │       │  • FiveWhys (Chuỗi 5-Why có số đo)     │
│  • HistoricalCases (Kho 30 ca tiền lệ)   │       │  • ReviewEvents (Lưu vết ai ký duyệt)  │
└──────────────────────────────────────────┘
└──────────────────────────────────────────┘
```

Nói bằng ngôn ngữ bình dân:

1. **Nhóm 1: Bảng "Bất di bất dịch" (Master Data)**:
   - Danh sách máy móc (`WorkCenters`), danh sách vật tư (`Materials`), danh mục lỗi chuẩn (`DefectCatalogues`).
   - *Ý nghĩa:* Đây là "khung xương thép". AI được gợi ý, nhưng mã máy và mã lỗi bắt buộc phải nằm trong danh mục này. Nếu AI dám chế ra mã máy "MILL-999" không có trong xưởng, hệ thống chặn lại ngay lập tức.
2. **Nhóm 2: Bảng "Hiện trường" (Operational Data)**:
   - Ghi nhận thực tế hôm nay xảy ra chuyện gì: Thông báo lỗi số mấy (`Notifications`), công nhân đo được bavia dày bao nhiêu mm (`Inspections`), đã nhốt bao nhiêu thùng hàng lỗi trong kho (`Containments`).
   - Mọi bảng này đều móc nối với nhau qua mã thông báo `notification_id` (tương đương phiếu QMEL trong SAP).
3. **Nhóm 3: Bảng "Quy trình 8 bước" (8D Entities)**:
   - Theo dõi tiến độ từ D1 đến D8: Đội đặc nhiệm gồm ai, biểu đồ xương cá Ishikawa gồm những nhánh nào, 5-Why ra sao, và ai là người bấm duyệt từng bước (`ReviewEvents`).
   - *Không có chuyện xóa dữ liệu:* Mọi hành động bấm nút duyệt, yêu cầu làm lại hay mở lại ca đều được lưu vết vĩnh viễn (Audit Trail).

1.3. Tại sao lại dùng SQLite mà không dựng PostgreSQL hay Cloud to đùng?

- **Tốc độ cực nhanh (In-process):** SQLite chạy ngay trong bộ nhớ của server, không tốn thời gian truyền mạng qua lại. Nhờ đó kiểm tra 4 testcase chỉ mất vài giây, truy vấn tiền lệ chưa tới 180ms.
- **Tiết kiệm chi phí & Không sợ sập mạng:** Nhà máy có thể chạy offline 100% trên một máy tính công nghiệp đặt ngay dưới xưởng.
- **Dễ chạy nghiệm thu (Zero Setup):** Ban Giám Khảo clone source code về chỉ cần gõ `npm start` là chạy ngay, không bắt BGK phải cài Docker hay cấu hình mật khẩu database phức tạp.

---

2. BẢN CHẤT AI SUY LUẬN — NÓ LÀM THẾ NÀO ĐỂ TỪ DỮ LIỆU RA KẾT QUẢ?

2.1. Nỗi sợ của BGK: "AI có đoán mò và bịa chuyện không?"

Câu trả lời của bạn: **"Không, vì chúng tôi trói AI bằng 2 chiếc vòng kim cô."**

Chiếc vòng 1: Bounded JSON Schema (Ép khuôn biểu mẫu)

- Người bình thường dùng ChatGPT là bảo: "*Hãy viết cho tôi một báo cáo sự cố*". Con bot sẽ tuôn ra một bài văn xuôi dài ngoằng, đúng sai lẫn lộn.
- Hệ thống của chúng ta **cấm tiệt văn xuôi tự do**. Toàn bộ lệnh gửi cho AI đều bị ép buộc phải trả về đúng cấu trúc JSON đã định sẵn:
  - Trường `measuredValue` bắt buộc phải là số thực (Float).
  - Trường `category` bắt buộc phải là 1 trong 6 nhánh xương cá: `Man`, `Machine`, `Material`, `Method`, `Measurement`, `Environment`.
  - Trường `isRootCause` chỉ được trả về `'Y'` hoặc `'N'`.
- Nếu AI trả về sai kiểu dữ liệu hoặc thiếu trường, code sẽ quăng lỗi và bắt model sinh lại ngay.

Chiếc vòng 2: First-Principles (Suy luận từ nguyên lý vật lý gốc)

- **Nguyên tắc vàng:** "Chỉ tin vào số đo vật lý lệch dung sai, không tin vào lời nói mồm."
- Ví dụ: Kỹ sư bảo "*Máy chạy bình thường, chắc do công nhân*".
  $ightarrow$ AI không nghe câu đó. AI đi tìm bảng `Inspections` và `Telemetry`:
  - Bản vẽ quy định độ bavia tối đa: `0.10 mm`.
  - Thước cặp đo thực tế: `0.26 mm`.
  - Lực kẹp dao: tụt từ `12 bar` xuống `9.5 bar`. $ightarrow$ AI kết luận: Mòn dao và tụt áp gá kẹp (nhánh Machine), dựa trên độ lệch dung sai `+0.16 mm` và `-2.5 bar`.

2.2. Luồng chạy 4 chặng (Execution Pipeline) ngoài đời thực

```
[Dữ liệu thô SAP QM bẩn]
        │
        ▼
┌──────────────────────────────────────────────────┐
│ CHẶNG 1: CODE 100% (< 10ms)                      │
│ • Cắt khoảng trắng thừa (' MAT-10247 ')          │
│ • Đổi dấu phẩy thập phân Đức ('0,32 mm' -> 0.32) │
│ • Đối chiếu khóa cứng Master Data                │
│ • CHẶN RÁC NGAY TỪ CỬA - KHÔNG TỐN 1 XU TIỀN AI  │
└──────────────────────────┬───────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────┐
│ CHẶNG 2: AI ENRICHMENT (~800ms)                  │
│ • Đọc đoạn văn mô tả tiếng Đức/tiếng Anh         │
│ • Bóc tách các số đo ẩn thành trường đo có cấu trúc │
│ • Rà soát lỗ hổng dữ liệu (Gaps Detection)       │
└──────────────────────────┬───────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────┐
│ CHẶNG 3: CHẨN ĐOÁN MÙ & GRAPH TRUY HỒI (~1,200ms) │
│ • AI Referee: BỊT MẮT AI, chỉ cho xem số đo thô  │
│ • Tự chẩn đoán độc lập để phát hiện định kiến đổ lỗi │
```

```
│ • GraphAI: Quét mạng lưới máy móc tìm tiền lệ tương tự │
└──────────────────────────┬──────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────┐
│ CHẶNG 4: SOẠN THẢO D1-D8 & SAFETY GATES (~1,500ms)   │
│ • Lên khung báo cáo 8 bước chuẩn công nghiệp         │
│ • Đề xuất hành động D5 kèm kiểm tra thực địa D6      │
│ • Đặt các cổng dừng an toàn (Stops) chờ kỹ sư duyệt  │
└──────────────────────────────────────────────────────┘
```

> **Tại sao không gọi 1 lần prompt cho xong mà phải chia 4 chặng?**
> *Giải thích dân dã:* Nếu gộp hết vào 1 prompt khổng lồ, AI sẽ mất 60–90 giây mới trả lời xong. Đem lên Cloud (Render, Vercel) là bị đứt kết nối (Timeout 504) ngay! Chia 4 chặng giúp mỗi chặng chạy 1 giây, tổng cộng chỉ mất ~4 giây, cực kỳ mượt mà và kiểm soát được lỗi ở từng chặng.

---

3. GRAPHAI LÀ GÌ? TẠI SAO VECTOR RAG THÔNG THƯỜNG LẠI THẤT BẠI?

3.1. Vector RAG thông thường hoạt động ra sao và vì sao nó "ngáo"?

- **Vector RAG truyền thống:** Nó biến câu văn thành một dãy số (vector) rồi so sánh độ tương đồng ngữ nghĩa câu chữ (Semantic Similarity).
- **Điểm chết người trong nhà máy:**
  - Ca sự cố 1: "*Máy phay CNC MILL-07 bị xước bề mặt chi tiết hợp kim nhôm*".
  - Ca sự cố 2: "*Máy tiện CNC LATHE-03 bị vết cào xước chi tiết thép tròn*".
  - Hai câu này về mặt ngữ nghĩa tiếng Việt/tiếng Anh thì giống hệt nhau ("xước", "máy CNC", "chi tiết").
  - Vector RAG thấy độ tương đồng cao (92%) nên bốc ngay giải pháp của *Máy tiện* (chỉnh dao tiện rãnh) đem sang áp dụng cho *Máy phay* (đang bị mòn đầu dao phay ngón).
    👉 **Hậu quả:** Gãy dao phay, hỏng cả lô hàng hàng trăm triệu đồng!

3.2. GraphAI giải quyết việc này như thế nào?

GraphAI không nhìn câu chữ bề mặt, mà nhìn vào **Mạng lưới quan hệ thực thể (Entity Knowledge Graph)**:

```
[TIỀN LỆ 8D-10048412]          [SỰ CỐ HIỆN TẠI]
         │                              │
         ├── Trùng Tổ Máy (WorkCenter) ─┤  -> Máy phay WC-MILL-07 (Đúng máy!)
         │                              │
         ├── Trùng Họ Vật Tư (Material) ┤  -> Nhôm đúc MAT-10247 (Đúng vật
liệu!)
         │                              │
         ├── Trùng Mã Lỗi (DefectCode) ─┤  -> Bavia cạnh DF-BURR-01 (Đúng lỗi!)
         │                              │
         ▼                              ▼
  [HÀNH ĐỘNG ĐÃ THÀNH CÔNG]      [ĐỀ XUẤT ÁP DỤNG]
```

```
  Thay đầu dao deburring          Thay đầu dao deburring EQ-MILL07-002
  và hiệu chuẩn lại đồ gá         và kiểm tra áp suất kẹp
```

Nói dễ hiểu: **GraphAI giống như việc tra gia phả lý lịch máy móc**:

- Nó có **5 đầu dò quan hệ**:
  1. `sameWorkCenter`: Xem chính cái máy này trước đây có từng bị bệnh này chưa (bệnh kinh niên mỏi cơ khí).
  2. `sameMaterialFamily`: Xem dòng hợp kim này có đặc tính ứng suất cắt giống nhau không.
  3. `sameDefectCode`: Đúng mã lỗi kỹ thuật ISO/VDA hay không.
  4. `resolvedByActionType`: Trước đây người ta đã chữa khỏi bệnh này bằng cách nào (thay dao, mài gá, hay đổi phôi).
  5. `GraphAnchor`: Mỏ neo đồ thị giúp bốc nhanh các ca liên quan mà không cần quét cạn database.

3.3. Quy trình 2 tầng lọc: Nhanh như chớp mà chuẩn như chuyên gia

- **Tầng 1 (Lọc thô trong 15ms):** Dùng công thức cộng điểm quy tắc (Trùng máy +4 điểm, trùng lỗi +4 điểm, trùng vật liệu +3 điểm) kết hợp khoảng cách vector Cosine. Lọc từ hàng trăm ca lấy ra **Top 10 ca sáng giá nhất**.
- **Tầng 2 (Lọc tinh bằng AI trong 150ms):** Đưa Top 10 ca này cho AI chấm điểm lại theo ngữ cảnh cơ chế vật lý hỏng hóc.
- **Bộ nhớ đệm LRU Cache (SHA-256):** Nếu cùng một lỗi đó mà người dùng bấm qua bấm lại giữa các bước D, hệ thống lấy ngay kết quả từ bộ nhớ đệm trong **< 10ms**, không tốn 1 xu tiền gọi API lặp lại.

---

4. AI REFEREE & CHẨN ĐOÁN MÙ (BLIND DIAGNOSIS) — VŨ KHÍ "ĂN TIỀN" NHẤT

4.1. Căn bệnh muôn thuở ở các nhà máy sản xuất: Thiên kiến đổ lỗi (Confirmation Bias)

- Khi có phế phẩm nổ ra, các kỹ sư xưởng thường chịu áp lực giao hàng rất lớn.
- Họ thường chọn con đường dễ nhất: **Đổ lỗi cho công nhân** (nhánh *Man* trong Ishikawa: "*do công nhân ca C làm ẩu*", "*do thao tác bất cẩn*", "*do chưa đọc kỹ hướng dẫn*").
- Công nhân bị trừ lương, bị phạt kỷ luật, nhưng tháng sau... **lỗi y như cũ lại nổ ra tiếp!** Vì nguyên nhân gốc rễ là cái máy bị rơ trục chứ không phải do con người.

4.2. AI Referee giải quyết thế nào bằng "Chẩn đoán mù"?

```
[Bản ghi sự cố của Kỹ sư]
  • Ghi chú: "Công nhân ca C gá lệch"
  • Ishikawa: Man (isRootCause = 'Y')
  • Đo kiểm: Rơ đầu dao 0.9mm (Max 0.2mm)
  • Lịch sử: Lỗi xuất hiện ở cả ca A, B, C
            │
            ▼
┌──────────────────────────────────────────────────────┐
│ BƯỚC 1: BỊT MẮT AI (blindEvidence.ts)                │
│ CẮT SẠCH kết luận "Công nhân ca C gá lệch"!          │
│ CẮT SẠCH cờ 'isRootCause' ở nhánh Man!               │
```

```
┌──────────────────────────────────────────────────────────┐
│ CHỈ GIỮ LẠI: Số đo 0.9mm và dữ liệu lỗi cả 3 ca A,B,C.  │
└──────────────────────────┬───────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────┐
│ BƯỚC 2: AI TỰ SUY LUẬN ĐỘC LẬP TỪ SỐ ĐO                  │
│ • Số đo rơ 0.9mm vượt trần cho phép 0.2mm.               │
│ • Cả 3 ca A, B, C đều bị -> Không thể do 1 người!        │
│ -> AI KẾT LUẬN: NGUYÊN NHÂN LÀ MÁY (MACHINE).            │
└──────────────────────────┬───────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────┐
│ BƯỚC 3: AI REFEREE TUÝT CÒI ĐỐI CHẤT (ADJUDICATION)     │
│ AI so sánh:                                              │
│   Kỹ sư nói: MAN (Không có số đo chứng minh)             │
│   AI thấy  : MACHINE (Bằng chứng số đo rơ 0.9mm)         │
│                                                          │
│ => AI TUÝT CÒI: KÍCH HOẠT CỔNG DỪNG D4!                  │
│ "Kỹ sư đổ lỗi cho người không có chứng cứ, trong khi     │
│ máy móc đang có số liệu bất thường. Khóa ca, gửi hồ sơ   │
│ lên Hội đồng Chất lượng để phân xử!"                     │
└──────────────────────────────────────────────────────────┘
```

👉 **Giá trị thực tế:** AI không làm vừa lòng con người. AI đóng vai trò như **ông trọng tài VAR trong bóng đá**, chỉ nhìn vào băng ghi hình (số liệu đo) để bảo vệ người công nhân khỏi bị kỷ luật oan và buộc nhà máy phải sửa đúng cái máy!

---

5. AI STOP LÀ GÌ? TẠI SAO LẠI CẮT BỎ TÍNH NĂNG "AI TỰ DUYỆT" (AUTO-APPROVE)?

5.1. Triết lý AI Stop: "Biết khi nào mình không biết là đỉnh cao của an toàn"

Một con AI nguy hiểm nhất là con AI "cái gì cũng gật đầu làm bừa".
Trong hệ thống của chúng ta, AI có **4 chiếc phanh khẩn cấp (4 Safety Stops)**:

Cổng dừng (Stop)	Khi nào thì đạp phanh?	Hệ thống xử lý ra sao?
1. `unverified-facts`	Số đo được bóc tách từ câu văn tự do, chưa ai xác nhận lại.	Dừng ở D4, bắt Kỹ sư kiểm tra xác nhận số đo thực tế mới cho đi tiếp.
2. `record-contradicts-evidence`	Kỹ sư đổ lỗi cho người mà không có số đo, trong khi máy móc lệch dung sai.	Dừng ở D4, tuýt còi thiên kiến xác nhận, yêu cầu điều tra lại máy móc.
3. `exceeds-authority`	Chi phí sự cố (COPQ) vượt trần thẩm quyền (> €10.000).	Dừng ở D8, bắt buộc Giám đốc chất lượng ký duyệt chữ ký số.
4. `outside-scope`	Lỗi hoàn toàn mới, trong kho 30 ca không có bất kỳ tiền lệ nào (Ví dụ	Dừng lại, **tuyệt đối từ chối bịa cách sửa**, sinh ngay gói 3 câu hỏi kỹ thuật

Cổng dừng (Stop)	Khi nào thì đạp phanh?	Hệ thống xử lý ra sao?
	xưởng phay cơ khí tự nhiên xuất hiện lỗi hàn laser).	chuyển giao cho Chuyên gia đầu ngành (SME).

5.2. Quyết định dũng cảm: Cắt bỏ tính năng "AI Auto-Approve"

- Ban đầu nhóm có thể lập trình cho AI tự bấm nút Approve từ D1 đến D8 để khoe là "tự động hóa 100%".
- Nhưng nhóm đã **chủ động cắt bỏ tính năng này**.
- **Lý do sống còn (Tiêu chuẩn ô tô IATF 16949 / VDA 6.3):**
  - Trong sản xuất ô tô, một linh kiện lỗi lọt ra ngoài có thể làm xe mất lái, gây tai nạn chết người và hãng xe phải triệu hồi hàng triệu xe thiệt hại hàng trăm triệu USD.
  - Khi tai nạn xảy ra, tòa án và cơ quan đăng kiểm sẽ hỏi: *"Ai là người phê duyệt cho lô hàng này xuất xưởng?"*
  - **AI không thể ngồi tù, AI không có tài khoản ngân hàng để đền tiền.** Trách nhiệm pháp lý bắt buộc phải gắn với chữ ký số của Kỹ sư trưởng!
  - Vì vậy: **AI chỉ làm 80% công việc soạn thảo, tính toán, làm trọng tài; 20% quyền quyết định cuối cùng luôn thuộc về con người.**

---

6. MÁY TRẠNG THÁI STEPGRAPH: CẤM ĐỐT CHÁY GIAI ĐOẠN

Quy chuẩn 8D công nghiệp rất nghiêm ngặt. Rất nhiều kỹ sư ẩu vừa thấy lỗi nổ ra đã vội vàng nhảy sang nghĩ cách sửa (D5).
Động cơ `StepGraph.ts` trong code khóa chặt tiến trình:

- **D1 & D2**: Phải lập đội và phân tích ranh giới lỗi (Cái gì bị / Cái gì KHÔNG BỊ) trước.
- **D3 (Khoanh vùng cách ly - Containment)**: **BẮT BUỘC PHẢI DUYỆT XONG MỚI ĐƯỢC SANG D4.** Không thể ngồi phòng máy lạnh tìm nguyên nhân khi hàng lỗi ở dưới xưởng vẫn đang tiếp tục chạy ra kho!
- **D4 (Nguyên nhân gốc)**: Bắt buộc xong mới được đề xuất giải pháp D5.
- **D6 & D7 (Thực nghiệm & Phòng ngừa)**: Phải có biên bản nghiệm thu tại hiện trường mới cho đóng ca D8.
- **Không có nút "Từ chối vứt xó" (`Rejected`):** Một sự cố chất lượng trong nhà máy không bao giờ được phép vứt vào thùng rác. Nó chỉ có trạng thái **Yêu cầu làm lại (`ChangeRequested`)** cho đến khi tìm ra đúng nguyên nhân gốc rễ thì thôi.

---

7. KỊCH BẢN THUYẾT TRÌNH BẰNG VĂN NÓI KHI ĐỨNG TRƯỚC BAN GIÁM KHẢO

Hãy tưởng tượng bạn đang cầm mic đứng trước BGK. Đừng đọc nguyên văn slide. Hãy nói một cách tự tin, chắc nịch theo dàn ý sau:

🗣️ Đoạn 1: Mở đầu gây ấn tượng (30 giây)

> *"Kính thưa Ban Giám Khảo, khi nhắc đến AI trong nhà máy, mọi người thường nghĩ đến việc dùng chatbot để viết văn. Nhưng trong ngành sản xuất công nghiệp, một câu văn hoa mỹ không giải quyết*

> *được phế phẩm.*
> *Đội thi UIT_4Lovpe mang đến **8D Copilot** — một hệ thống Trợ lý Thẩm định & Trọng tài Chất lượng xây dựng chuẩn hóa theo SAP QM, suy luận từ số đo vật lý thực tế và truy hồi tiền lệ theo đồ thị thực thể GraphAI."*

🗣️ Đoạn 2: Thuyết trình 3 điểm kỹ thuật đắt nhất (2 phút)

> *"Điểm cốt lõi thứ nhất: **Chống ảo giác tuyệt đối bằng dữ liệu**. Toàn bộ Master Data mã máy, mã linh kiện của SAP được khóa cứng. Đầu ra của AI bị ép khuôn 100% bằng JSON Schema và nguyên lý First-Principles. Muốn kết luận mòn dao, bắt buộc phải có số đo bavia lệch dung sai chứ không chấp nhận suy đoán mồm.
>
> Điểm cốt lõi thứ hai: **GraphAI thông minh hơn Vector RAG**. Vector RAG chỉ so chữ với chữ nên rất dễ lấy nhầm cách sửa máy tiện áp dụng cho máy phay. GraphAI của chúng tôi kết nối 5 đầu dò thực thể: cùng máy, cùng họ nhôm thép, cùng mã lỗi ISO và tiền lệ FMEA, bảo đảm tìm đúng gốc rễ trong chưa đầy 180ms.
>
> Điểm cốt lõi thứ ba cũng là điểm ăn tiền nhất: **AI Referee phản biện định kiến con người**. Kỹ sư thường có thói quen đổ lỗi cho công nhân. Nhưng AI của chúng tôi được bịt mắt hoàn toàn: nó xóa hết kết luận của kỹ sư, tự đọc số đo cảm biến thô. Khi phát hiện kỹ sư đổ lỗi cho người mà không có chứng cứ trong khi máy móc lệch 0.9mm, AI sẽ tuýt còi khóa ca lại ngay để bảo vệ sự thật!"*

🗣️ Đoạn 3: Kết bài đĩnh đạc (30 giây)

> *"Cuối cùng, chúng tôi chủ động cắt bỏ tính năng AI tự động phê duyệt (Auto-Approve). Trong tiêu chuẩn an toàn ô tô IATF 16949, sinh mạng con người là trên hết, chữ ký phê duyệt bắt buộc phải thuộc về Kỹ sư trưởng.*
> *8D Copilot: Công nghệ thực dụng — Giá trị bền vững. Tiết kiệm 90% thời gian lập hồ sơ, bảo vệ người công nhân khỏi định kiến oan và sẵn sàng tích hợp ngay vào dây chuyền sản xuất lớn. Xin cảm ơn Ban Giám Khảo!"*

---

8. BỘ 5 CÂU HỎI "HÓC BÚA" CỦA BAN GIÁM KHẢO VÀ CÁCH ĐỐI ĐÁP

❓ Q1: "Tại sao phải làm GraphAI làm gì cho phức tạp, sao không dùng Vector RAG cho nhanh?"

👉 Bạn trả lời:
"Dạ thưa Giám khảo, Vector RAG chỉ tìm kiếm theo sự tương đồng ngữ nghĩa câu từ. Trong xưởng máy, hai câu 'máy tiện bị xước' và 'máy phay bị xước' câu từ giống nhau 90%, Vector RAG sẽ bốc nhầm cách sửa của máy tiện đè sang máy phay làm gãy dao. GraphAI của chúng tôi tra cứu theo mạng lưới thực thể: đúng tổ máy, đúng đồ gá, đúng mã hợp kim và lịch sử FMEA thì mới lấy tiền lệ, nên độ chính xác kỹ thuật là tuyệt đối ạ."

❓ Q2: "Làm sao các bạn dám chắc 100% là AI không bịa đặt số liệu (Hallucination)?"

👉 Bạn trả lời:
*"Dạ, chúng tôi dùng 3 tầng phòng thủ:

1. **Ép khuôn JSON Schema:** Cấm AI sinh văn bản tự do, ép kiểu số thực và enum nghiêm ngặt.

2. **First-Principles:** Mọi nguyên nhân trong 5-Why bắt buộc phải gắn với số đo cảm biến lệch dung sai, không có số đo thì bị đánh dấu cảnh báo.
3. **Bộ lọc Whitelist:** Sau khi AI sinh xong, code tự động rà lại với Master Data của SAP, hễ có mã máy hay vật tư lạ là hủy bỏ ngay lập tức."*

❓ Q3: "Hệ thống này có chạy Offline trong nhà máy được không, hay cứ phải có Internet gọi OpenAI/DeepSeek?"

👉 Bạn trả lời:
*"Dạ thưa Giám khảo, hệ thống hoàn toàn chạy Offline on-premise được ạ:

- Toàn bộ 16 bảng dữ liệu chạy In-Process bằng SQLite cực nhẹ.
- Chặng 1 chuẩn hóa dữ liệu chạy bằng code thuần 100%.
- Hệ thống hỗ trợ cơ chế Composite LlmProvider: có thể trỏ thẳng vào mô hình nội bộ chạy bằng Ollama hoặc LM Studio dưới xưởng mà không cần một giọt Internet nào."*

❓ Q4: "Cơ chế Chẩn đoán mù (Blind Diagnosis) của các bạn hoạt động thực tế như thế nào?"

👉 Bạn trả lời:
"Dạ, trong file `blindEvidence.ts`, khi nhận hồ sơ vào, code của chúng tôi dùng hàm filter cắt sạch kết luận của kỹ sư, xóa luôn biểu đồ xương cá và chuỗi 5-Why của con người. Chúng tôi chỉ ném cho AI các con số đo kiểm thô từ thước cặp và cảm biến. AI tự suy luận độc lập ra kết quả của nó. Sau đó mới đem kết quả của AI đối chiếu với kết luận của kỹ sư. Nếu kỹ sư đổ lỗi cho công nhân mà không có số đo trong khi cảm biến báo máy bị rơ, AI Referee sẽ tuýt còi Stop D4 để chặn đứng định kiến cảm tính ạ."

❓ Q5: "Tại sao các bạn không làm tính năng AI tự động bấm duyệt (Auto-Approve) để đạt 100% tự động hóa?"

👉 Bạn trả lời:
"Dạ, đây là quyết định thiết kế có chủ đích của nhóm sau khi nghiên cứu kỹ tiêu chuẩn ô tô IATF 16949 và VDA 6.3. Một quyết định nghiệm thu hành động D6 hay đóng ca D8 liên quan trực tiếp đến tính mạng người lái xe và trách nhiệm pháp lý hàng triệu USD. AI không thể chịu trách nhiệm pháp lý trước tòa án. Vì vậy, AI chỉ làm 80% công việc tính toán, làm trợ lý và trọng tài; 20% quyền bấm nút phê duyệt cuối cùng bắt buộc phải thuộc về Kỹ sư trưởng ạ."

(Tài liệu lưu hành nội bộ — Đội thi UIT_4Lovpe — MLAI Hackathon 2026)